"Be steadfast, immovable, always abounding in the work of the Lord." **— 1 Corinthians 15:58**

Supervised Learning Classification: Logistic Regression Classifiers

Course Instructor: Prasanna Ranjith Christodoss
Associate Professor of Comp. Sc., Messiah University, PA 17055

Mark Fenner, ***Introduction to Machine Learning with Python***
Miroslav Kubat,
An Introduction to Machine Learning

Lecture 10

Lecture Outline

1. What is Logistic Regression?
2. The Logit Function
3. Probability, Odds & Log-Odds
4. The Logistic Regression Model
5. Steps of Logistic Regression
6. Binary Classification: Worked Example
7. Python Implementation
8. Making Predictions
9. Interpreting Results

Key Goal

Predict a binary outcome (0/1) from input features using probability.

wisdom expertise

What is Logistic Regression?

Definition

A **classification algorithm** that predicts the **probability** of a binary outcome (0 or 1).

Despite the name "regression," it is used for ***classification*** tasks.

Focus: Binary classification (Y = 0 or Y = 1)

Stage 1: Compute Probabilities

Calculates probability of each class from input features.

Stage 2: Assign Class

Assigns the example to the class with the highest probability.

The Logit Function

Core Idea:

Instead of predicting Y directly, logistic regression predicts a ***transformation of Y*** called the **logit.**

The Problem

Linear regression can predict values outside [0, 1]
not valid as probabilities.

The Problem: “Over-Predictor”

Imagine you are trying to predict the probability that a student passes an exam based on how many hours they study.

- **Linear Regression** tries to draw a straight line through the data.
- **The Issue:** If someone studies for 100 hours, a linear model might predict a "120% chance of passing." In the real world, a probability cannot be higher than 100% (1.0) or lower than 0% (0.0).

Analogy: It’s like trying to fit a 12-inch ruler into a 10-inch box. It just doesn't fit the boundaries of reality.

The Logit Function

Core Idea:

Instead of predicting Y directly, logistic regression predicts a ***transformation of Y*** called the **logit.**

The Problem

Linear regression can predict values outside [0, 1] — not valid as probabilities.

The Logit

$$logit(p) = \ln\left(\frac{p}{1-p}\right)$$

Maps probability [0,1] → any real number (−∞, +∞). Can be modeled linearly.

$\left(\frac{p}{1-p}\right)$ is the "Odds"

If 80% Rain Predicted, the odds are $\frac{0.8}{0.2} = 4$

If probability p = 0.5, the logit is ln(1) = 0.
As p gets closer to 1, the logit goes toward +∞.
As p gets closer to 0, the logit goes toward - ∞.

The Logit Function

Core Idea:

Instead of predicting Y directly, logistic regression predicts a ***transformation of Y*** called the **logit.**

The Problem

Linear regression can predict values outside [0, 1] — not valid as probabilities.

The Logit

$$logit(p) = \ln\left(\frac{p}{1-p}\right)$$

Maps probability [0,1] → any real number (−∞, +∞). Can be modeled linearly.

$\left(\frac{p}{1-p}\right)$ is the "Odds"

If 80% Rain Predicted, the odds are $\frac{0.8}{0.2} = 4$

Back to Probability

$$p = \frac{1}{1 + e^{-logit}}$$

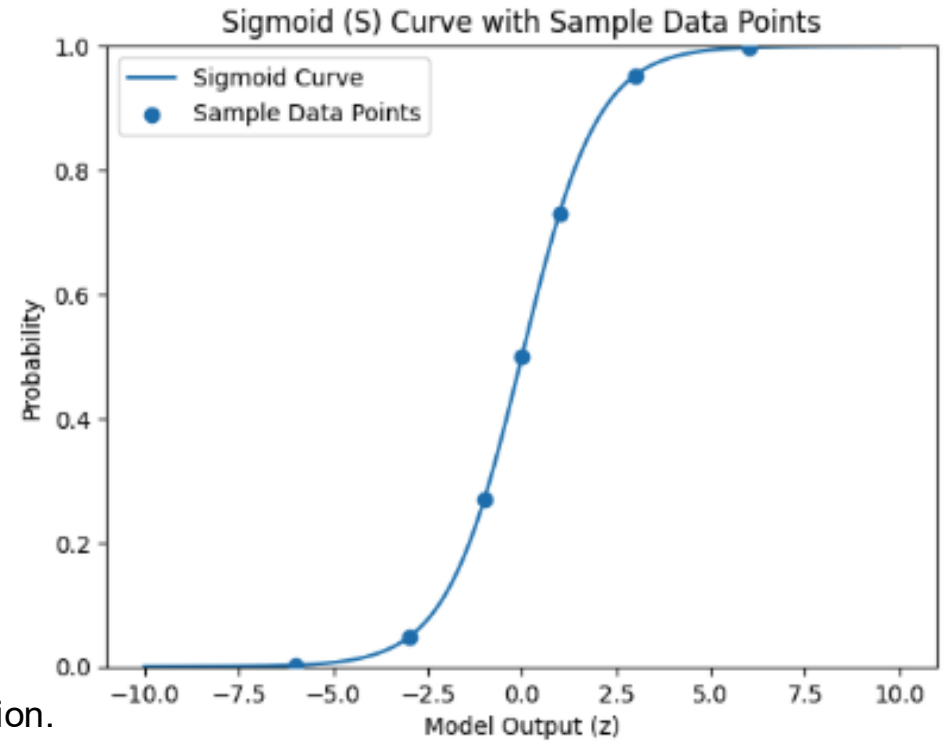
The sigmoid function maps logit back to a probability in [0, 1].

- **The "S" Curve:** This function takes any number (no matter how big or small) and squashes it back into a beautiful **S-shaped curve** that stays strictly between 0 and 1.
- **Result:** Even if our model calculates a massive value for a 100-hour studier, the Sigmoid function will cap the result at 0.999… (essentially 100%).

The Logit Function

Core Idea:

Instead of predicting Y directly, logistic regression predicts a ***transformation of Y*** called the **logit.**



The **smooth S-shaped line** is the Sigmoid function.

- Points on the **left** (large negative values) → probability close to **0**
- Points near **0** → around **50% probability**
- Points on the **right** (large positive values) → probability close to **1**

Probability, Odds & Log-Odds

Probability P(E)	Odds = P / (1−P)	Log-Odds = ln(Odd
0% (0.00)	0.00	−∞
10% (0.10)	0.111	−2.197
25% (0.25)	0.333	−1.099
50% (0.50)	1.000	0.000
75% (0.75)	3.000	+1.099
90% (0.90)	9.000	+2.197
100% (1.00)	∞	+∞

Key Insight

When Prob = 50%, Odds = 1 and Log-Odds = 0

Positive Log-Odds → event is likely

Negative Log-Odds → event is unlikely

The logit (log-odds) is what logistic regression actually models as a linear function.

The Logistic Regression Model

$$\text{logit}(p) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_q X_q$$

logit(p)

Log-odds of the event (class 1). The outcome we model linearly.

p

Probability of belonging to class 1. Ranges from 0 to 1.

β_0

Intercept term: the baseline log-odds when all predictors are 0.

$\beta_1 \ldots \beta_q$

Coefficients for each feature. Show how much each feature shifts the log-odds.

$X_1 \ldots X_q$

Input features (predictor variables).

Sigmoid

p = 1 / (1 + e^(−logit)) converts log-odds back to probability [0,1].

wisdom expertise

Steps of Logistic Regression

1

Estimate the Logit Function

Find the coefficients (β values) that minimize the classification error using Maximum Likelihood Estimation (MLE).

2

Compute Probability

Apply the sigmoid function to the logit output: $p = 1 / (1 + e^{-logit})$. This maps any real value to a probability between 0 and 1.

3

Apply Probability Cut-Off to Predict Class

If $p \geq 0.5 \rightarrow$ predict Class 1
If $p < 0.5 \rightarrow$ predict Class 0
The threshold (0.5) can be adjusted based on the problem.

Binary Classification Example

Scenario: Predict whether a student passes an exam based on hours studied and hours slept.

Hours Studied	Hours Slept	Passed? (Y)
2	8	**0 — Fail**
3	7	**0 — Fail**
4	6	**0 — Fail**
5	7	**1 — Pass**
6	8	**1 — Pass**
7	9	**1 — Pass**
8	9	**1 — Pass**

Observations

Binary outcome: Y = 0 (Fail) or Y = 1 (Pass)

Two features: X_1 = Hours Studied, X_2 = Hours Slept

Pattern: Students who study ≥ 5 hours tend to pass.

Goal: Build a model to predict pass/fail for new students.

Python Implementation

Python

```python
import pandas as pd
from sklearn.linear_model import LogisticRegression

# Dataset
data = {
    'Hours_Studied': [2, 3, 4, 5, 6, 7, 8],
    'Hours_Slept':   [8, 7, 6, 7, 8, 9, 9],
    'Passed':        [0, 0, 0, 1, 1, 1, 1]
}
df = pd.DataFrame(data)

# Features and target
X = df[['Hours_Studied', 'Hours_Slept']]
y = df['Passed']

# Train logistic regression model
model = LogisticRegression()
model.fit(X, y)

# View learned parameters
print("Coefficients:", model.coef_)
print("Intercept:",    model.intercept_)
```

Step 1

Build a DataFrame with features (X) and target (y).

Step 2

Create a LogisticRegression object and call .fit() to train.

Step 3

Inspect .coef_ (β_1, β_2) and .intercept_ (β_0).

Model Output & Interpretation

```
Coefficients: [[1.082,  0.391]]
Intercept:    [-7.621]
```

$$\text{logit}(p) = -7.621 + 1.082 \cdot X_1 + 0.391 \cdot X_2$$

1.082

Hours Studied (X_1)

Each extra hour studied increases log-odds of passing by 1.082, the strongest predictor.

0.391

Hours Slept (X_2)

Each extra hour slept increases log-odds by 0.391, a smaller but positive effect.

−7.621

Intercept (β_0)

Baseline log-odds when both features are 0. A large negative value means the prior is to fail.

Making Predictions

Problem: A student studied 4 hours and slept 8 hours. Will they pass?

$$\text{logit}(p) = -7.621 + 1.082 \cdot X_1 + 0.391 \cdot X_2$$

Making Predictions

Problem: A student studied 4 hours and slept 8 hours. Will they pass?

Step 1: Compute Log-Odds

```
log_odds = -7.621 + (1.082 × 4) + (0.391 × 8)
         = -7.621 + 4.328 + 3.128
         = -0.165
```

Step 2: Apply Sigmoid

```
p = 1 / (1 + e^(0.165))  =  0.459  (45.9%)
```

Step 3: Apply Threshold

```
p = 0.459  <  0.5  →  Predicted Class = 0  (FAIL)
```

Prediction

FAIL

Probability of passing: **45.9%**

Below the 0.5 threshold → classified as Fail.

Result Interpretation

Log-Odds **−0.165**	Slightly negative log-odds. The evidence leans slightly toward failing (class 0). Negative log-odds means the probability of the event is below 50%.
Probability **45.9%**	The model estimates a 45.9% chance of passing. This is close to the decision boundary (50%), indicating uncertainty in the prediction.
Predicted Class **FAIL (0)**	Since 0.459 < 0.5 (threshold), the model predicts FAIL. If the student had studied one more hour (5 hrs), the model would likely predict PASS.

wisdom expertise

Summary

✓ Logistic Regression is a Classifier

Despite "regression" in the name, it is used for binary (and multi-class) classification.

✓ The Logit Links Features to Probability

$\text{logit}(p) = \beta_0 + \beta_1 X_1 + \ldots + \beta q Xq$. The sigmoid maps this to a [0,1] probability.

✓ Three Core Steps

Estimate logit → Compute probability via sigmoid → Apply threshold to classify.

✓ Coefficients Have Meaning

Each β_i represents the change in log-odds per unit increase in feature X_i.

✓ Easy with scikit-learn

LogisticRegression().fit(X, y) trains the model. Use .predict() and .predict_proba() for inference.